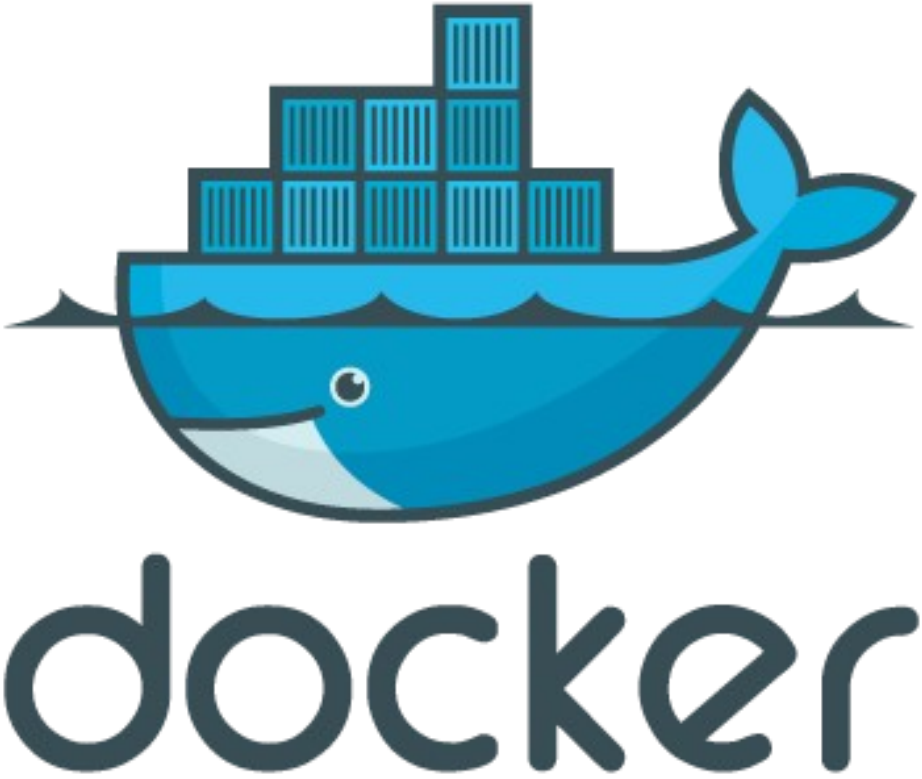
SISTEMAS DE GESTIÓN EMPRESARIAL

UD01 · Introducción a Docker

Documentación de las actividades prácticas



Alumno: Alan Souza Costa

Ciclo formativo: Desarrollo de Aplicaciones Multiplataforma (DAM)

Entorno de trabajo: Lubuntu en una máquina virtual Oracle VirtualBox

Repositorio de GitHub: https://github.com/l4nin/SGE

Fecha: 4 de noviembre de 2026

Índice

1. Introducción y objetivos

2. Entorno de trabajo

3. Actividad 01. Repositorio de GitHub

4. Actividad 02. Prácticas con Docker

   4.1. Instalación de Docker (Unidad 2)

   4.2. Comandos básicos: servidor Nginx

   4.3. Docker Compose: WordPress y MySQL (Unidad 6, caso práctico 1)

5. Resultados y conclusiones

6. Actividad 03. Foro

1. Introducción y objetivos

En esta actividad se ha trabajado con Docker para instalar y ejecutar aplicaciones mediante contenedores. El objetivo es aprender a desplegar servicios en un entorno aislado, comprobar su funcionamiento desde el navegador y gestionarlos mediante los comandos habituales de Docker y Docker Compose.

Se han realizado las actividades obligatorias indicadas en el enunciado. Se han dejado fuera las actividades opcionales de la Unidad 3, la Unidad 5 y el caso práctico 4 de la Unidad 6, tal y como se solicitó.

2. Entorno de trabajo

Las pruebas se han realizado en una máquina virtual con Lubuntu, ejecutada mediante Oracle VirtualBox. Docker se ha instalado y se ha gestionado desde el terminal.

La versión de Docker comprobada durante la práctica es la 29.9.2. También se ha verificado que el servicio docker estaba activo y habilitado para iniciarse con el sistema.

3. Actividad 01. Repositorio de GitHub

Para organizar los archivos y las prácticas de la asignatura se ha utilizado el siguiente repositorio:

https://github.com/l4nin/SGE

La captura aportada muestra el repositorio SGE de la cuenta l4nin, con varias carpetas de actividades y un archivo README.md. En la captura, el repositorio aparece como público. El enunciado solicita que sea privado y que se comparta con el usuario oscgaresc. Por tanto, antes de entregar la actividad hay que cambiar su visibilidad a privada y comprobar que oscgaresc aparece como colaborador.

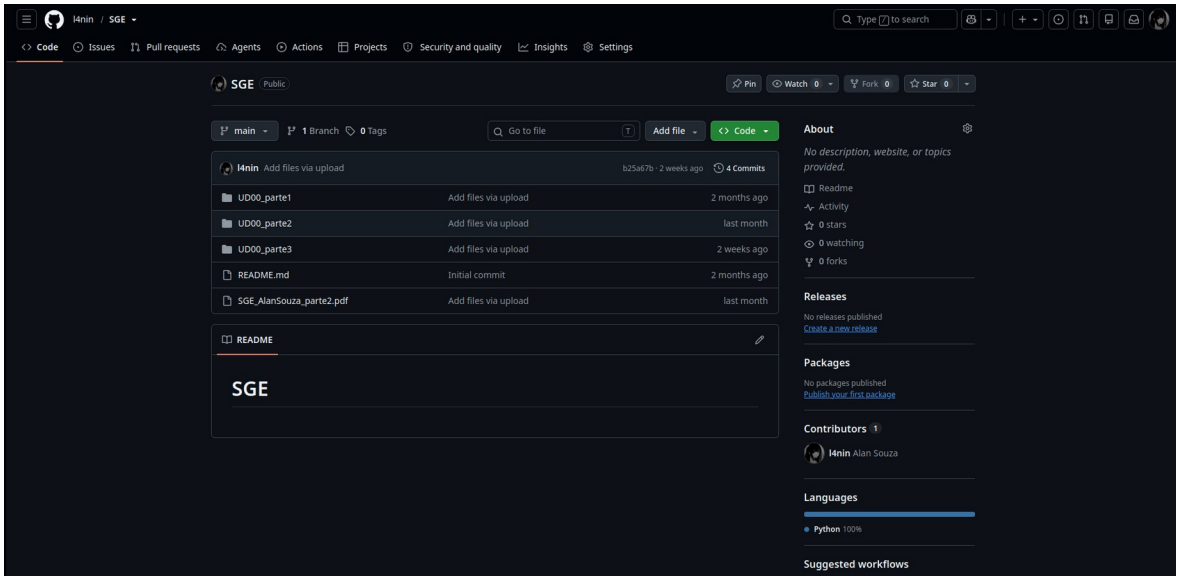


Figura 1. Repositorio SGE en GitHub.

4. Actividad 02. Prácticas con Docker

4.1. Instalación de Docker (Unidad 2)

Docker se ha instalado en la máquina virtual con Lubuntu. Después de la instalación, se ha consultado la versión mediante el comando docker --version y se ha comprobado el estado del servicio con systemctl. Por último, se ha ejecutado la imagen hello-world para verificar que Docker podía descargar una imagen e iniciar un contenedor correctamente.

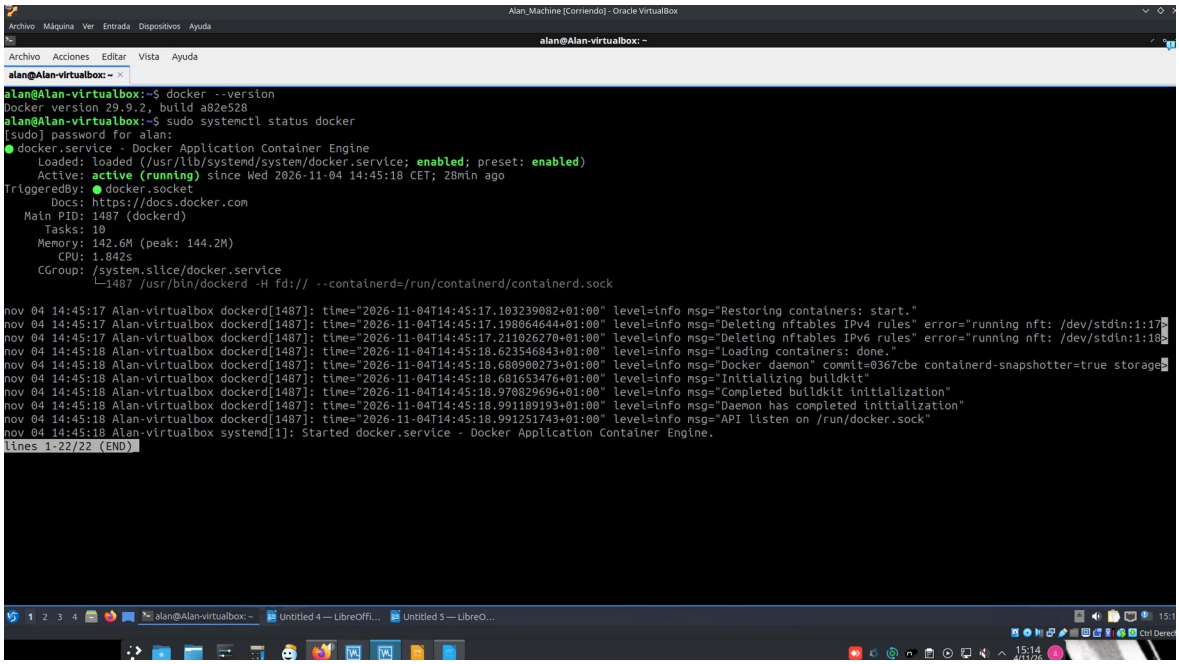


Figura 2. Comprobaciones iniciales de Docker.

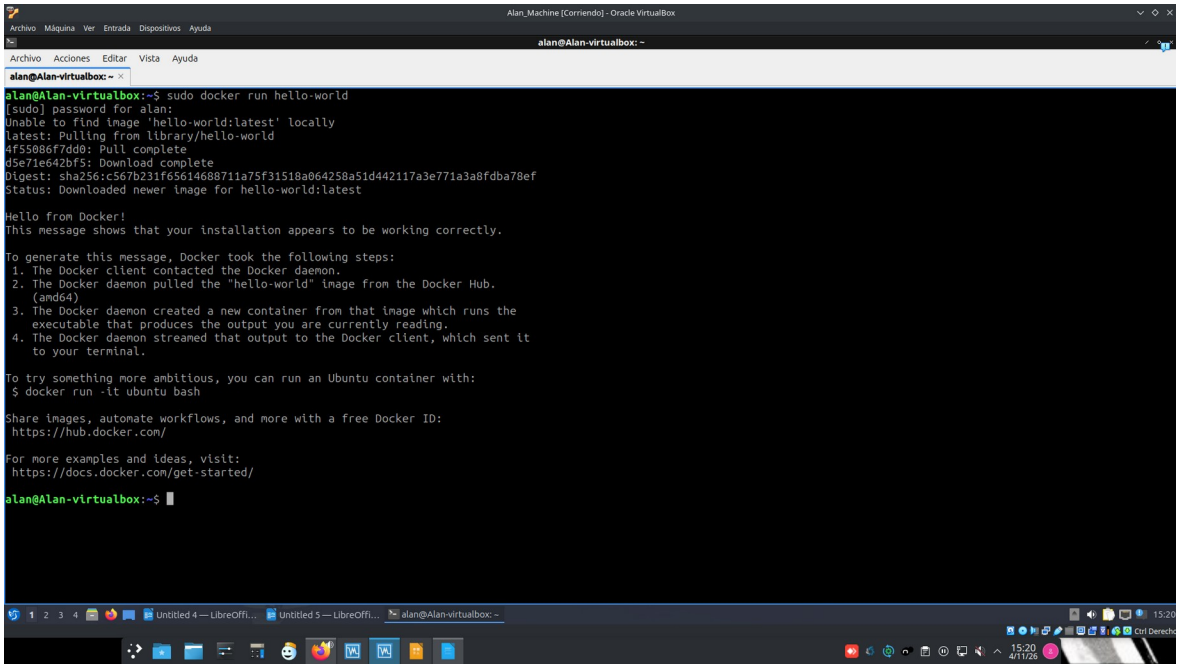


Figura 3. Ejecución de la imagen hello-world.

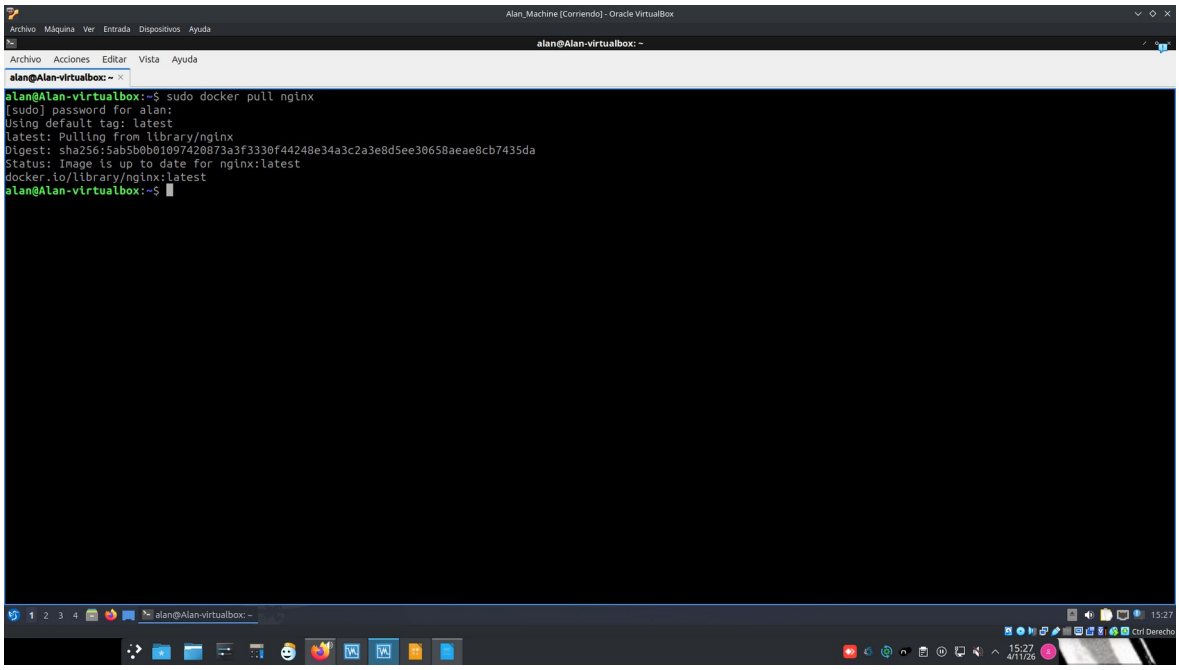


Figura 4. Resultado de las comprobaciones de Docker.

4.2. Comandos básicos: servidor web Nginx

En primer lugar, se ha descargado la imagen oficial de Nginx mediante el comando sudo docker pull nginx. A continuación, se ha creado y ejecutado el contenedor en segundo plano, asignándole el nombre servidor-practica y conectando el puerto 8080 del equipo con el puerto 80 del contenedor:

```
sudo docker run -d --name servidor-practica -p 8080:80 nginx
```

Se ha comprobado que el contenedor estaba en ejecución mediante docker ps. Después, se ha abierto Firefox y se ha accedido a http://localhost:8080. El navegador ha mostrado el mensaje «Welcome to nginx!», lo que confirma que el servidor web funcionaba correctamente. Para terminar, se ha detenido y eliminado el contenedor con los comandos sudo docker stop servidor-practica y sudo docker rm servidor-practica.

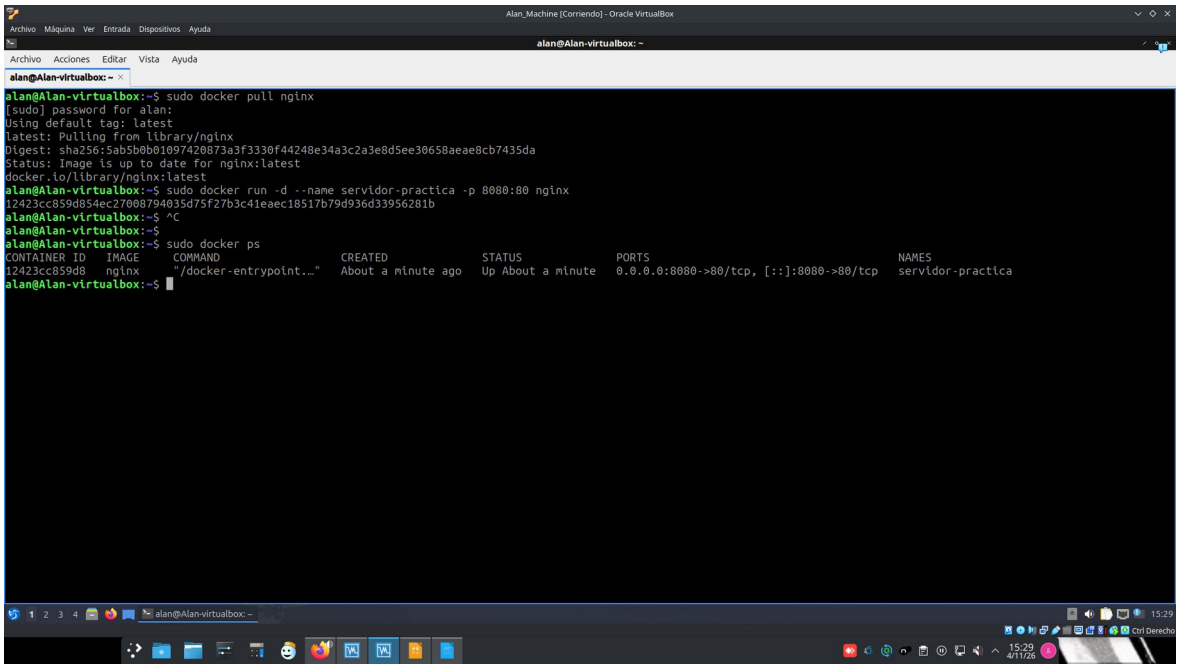


Figura 5. Descarga y ejecución del contenedor Nginx.

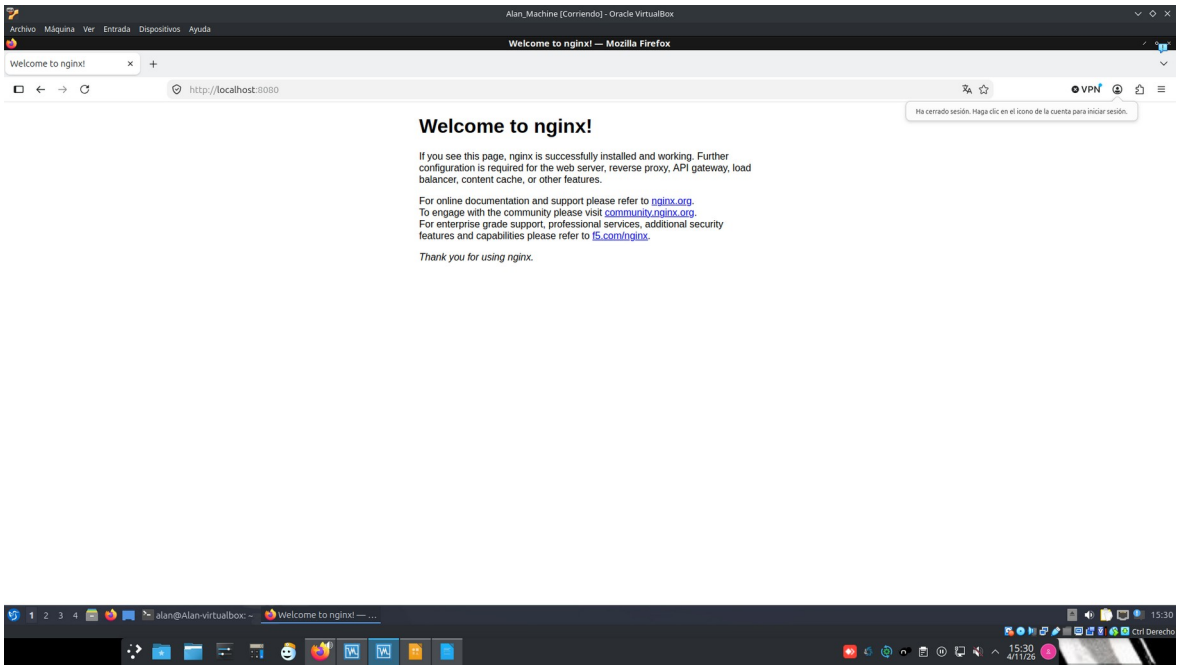


Figura 6. Página de bienvenida de Nginx en el navegador.

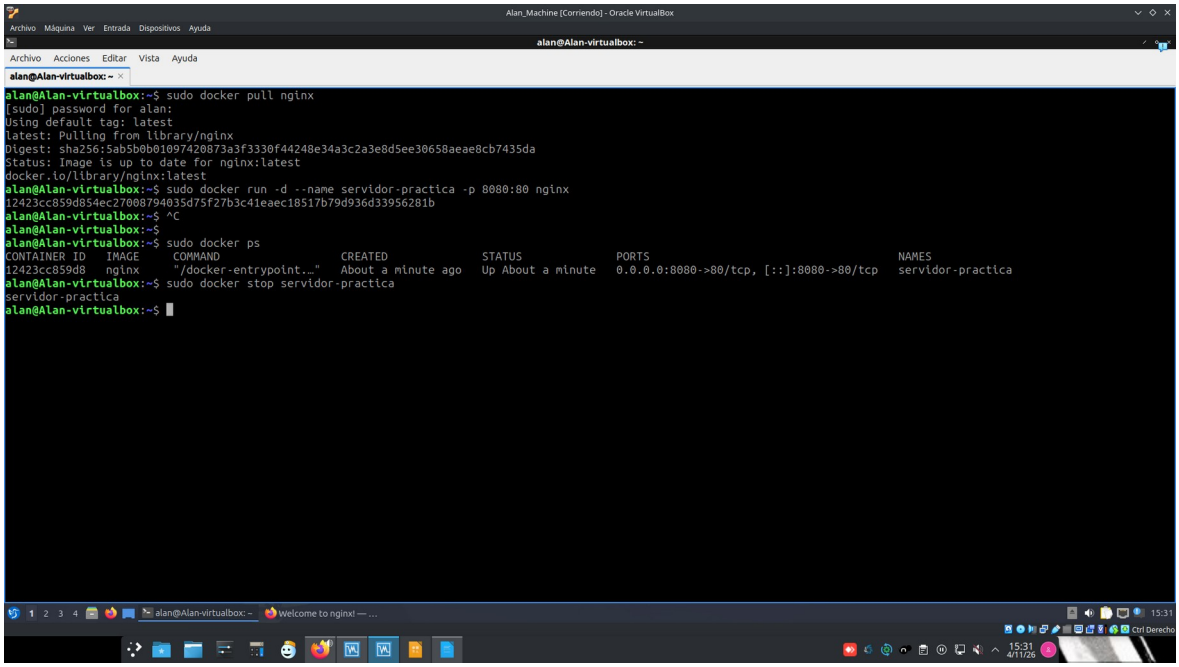


Figura 7. Comprobación del contenedor Nginx.

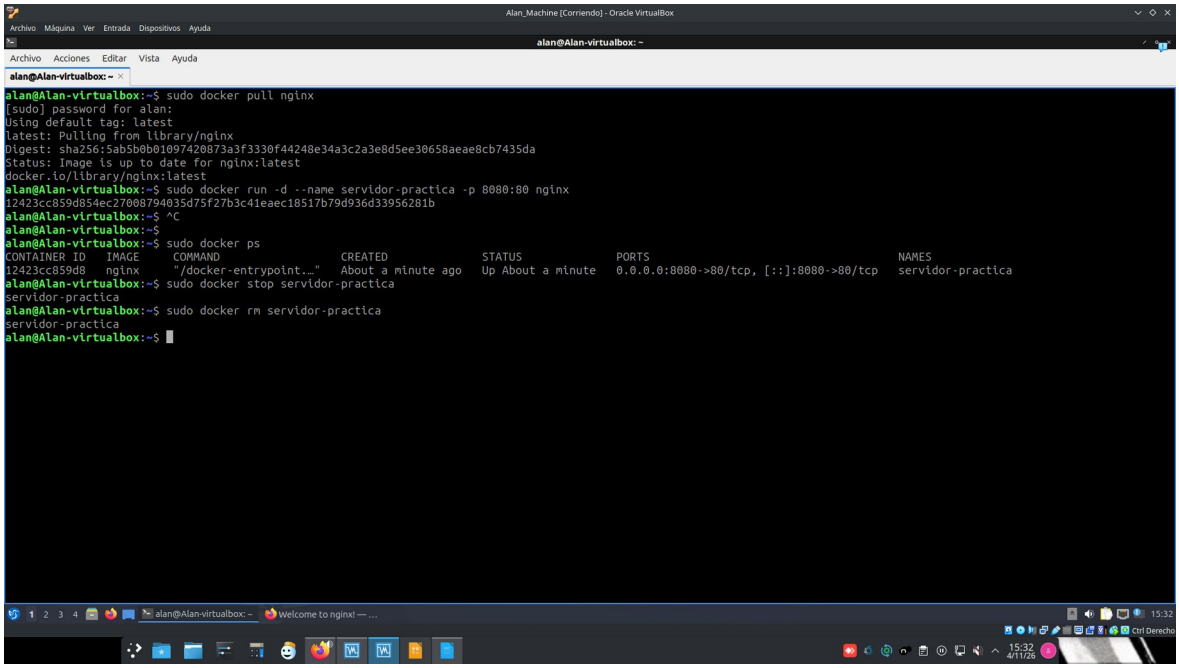


Figura 8. Detención y eliminación del contenedor.

4.3. Docker Compose: WordPress y MySQL (Unidad 6, caso práctico 1)

Se ha creado el directorio ~/practica-compose y, dentro de él, el archivo compose.yaml. La configuración define dos servicios: db, basado en la imagen mysql:8.0, y wordpress, basado en la imagen wordpress:latest. El servicio WordPress depende del servicio de base de datos y publica el puerto 80 del contenedor en el puerto 8080 del equipo.

El servicio MySQL utiliza las variables MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD y MYSQL_ROOT_PASSWORD. WordPress utiliza WORDPRESS_DB_HOST,

WORDPRESS_DB_NAME, WORDPRESS_DB_USER y WORDPRESS_DB_PASSWORD para conectarse a MySQL. También se ha definido el volumen persistente db_data para conservar los datos de la base de datos.

```
Comandos utilizados:
 sudo docker compose config
 sudo docker compose up -d
 sudo docker compose ps
 sudo docker compose logs db
 sudo docker compose logs wordpress
 sudo docker compose down
```

Durante las primeras pruebas, WordPress mostraba un error de conexión con la base de datos porque faltaba configurar la variable WORDPRESS_DB_HOST. Se añadió la dirección db:3306 en el servicio wordpress y se volvieron a iniciar los servicios. Tras corregir la configuración, la instalación de WordPress se completó correctamente. Se comprobó el acceso al panel de administración y a la página pública desde http://localhost:8080.

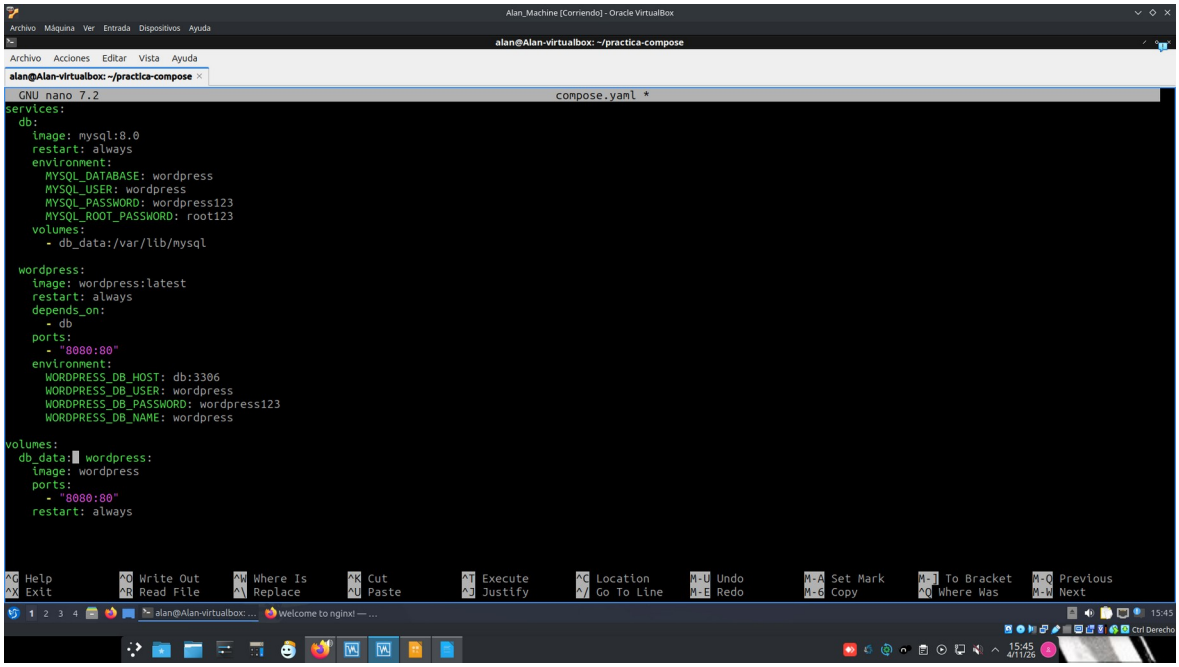


Figura 9. Configuración de los servicios en Docker Compose.

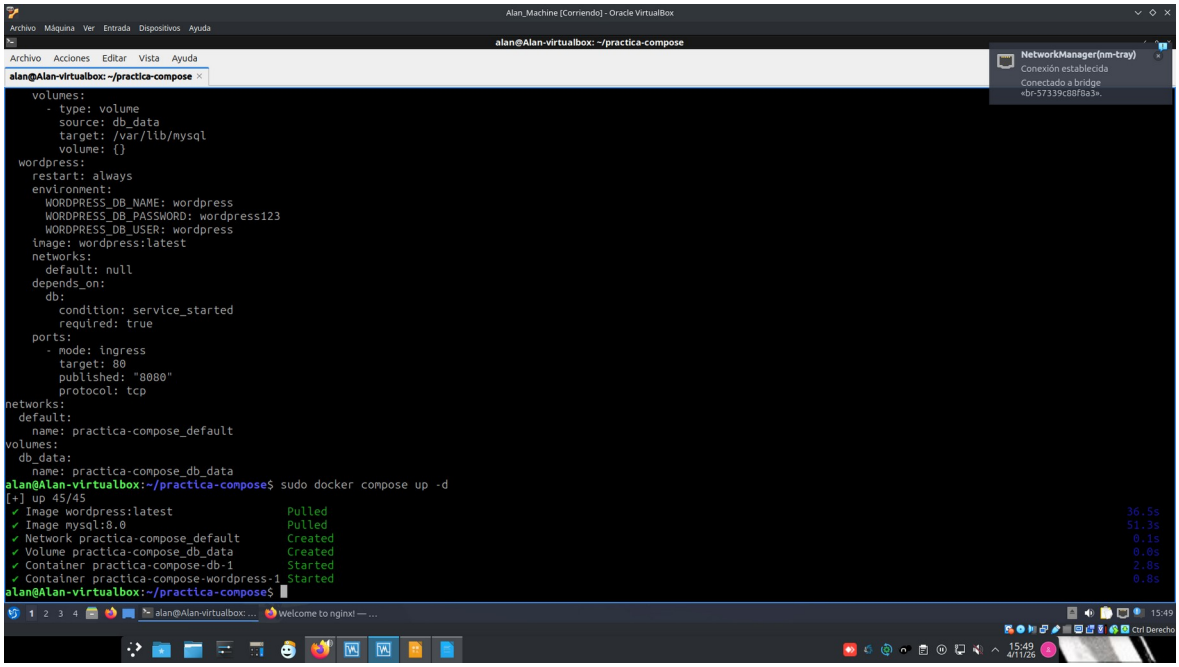


Figura 10. Inicio de los servicios de WordPress y MySQL.

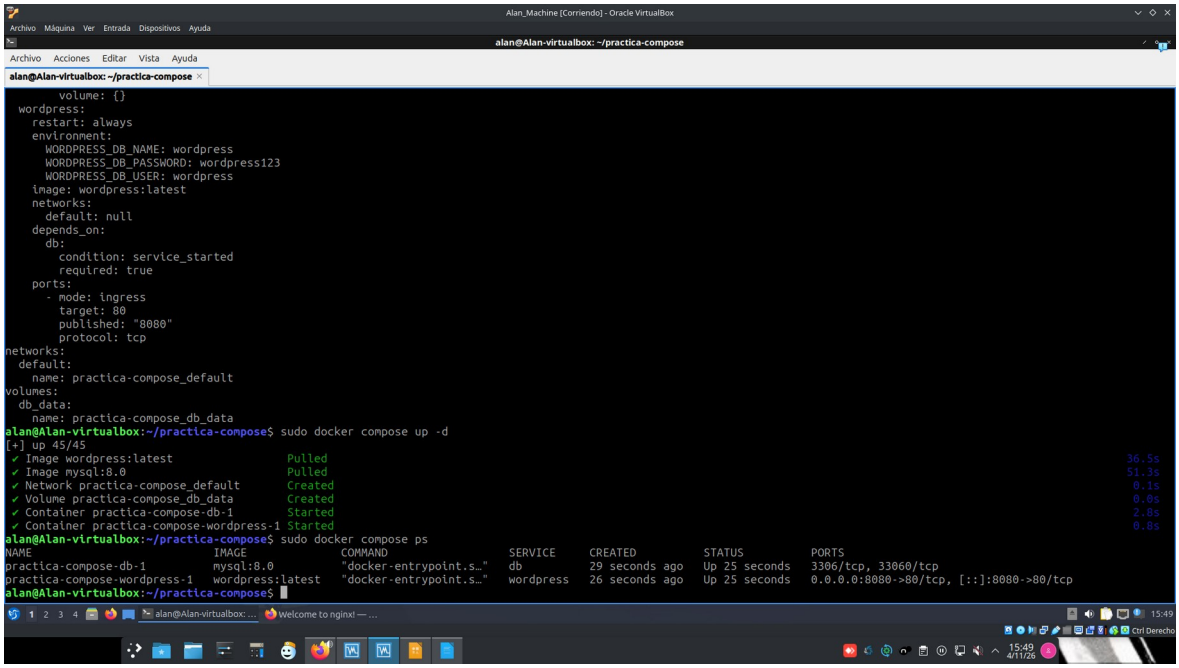


Figura 11. Comprobación del estado de los contenedores.

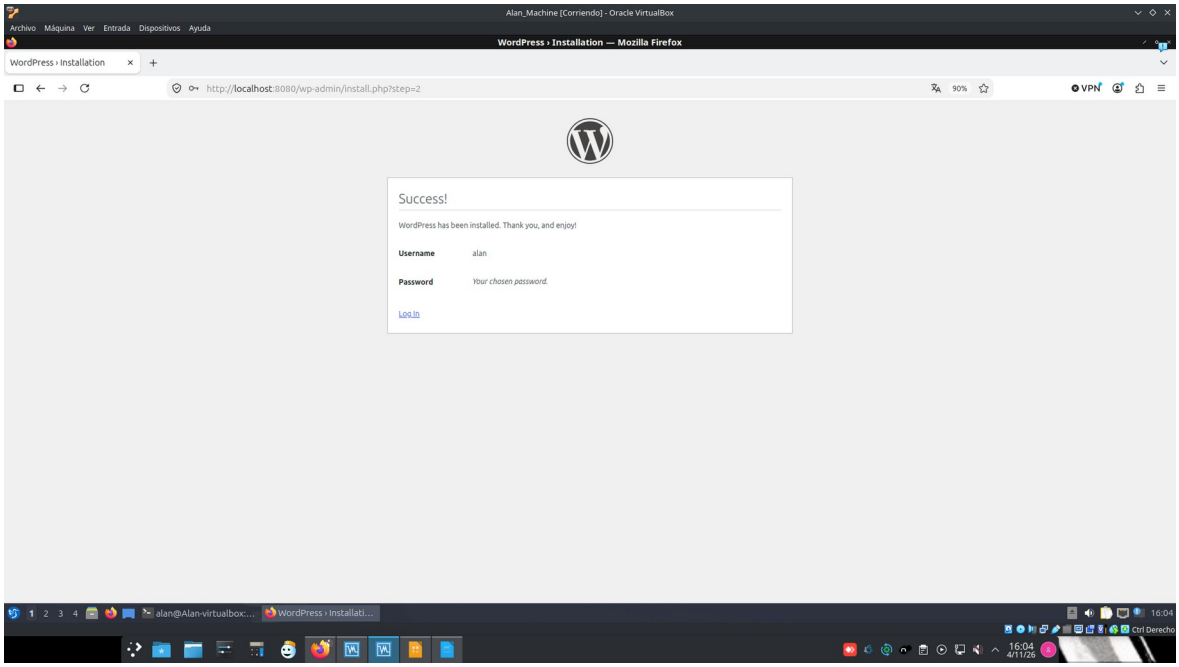


Figura 12. Instalación de WordPress completada.

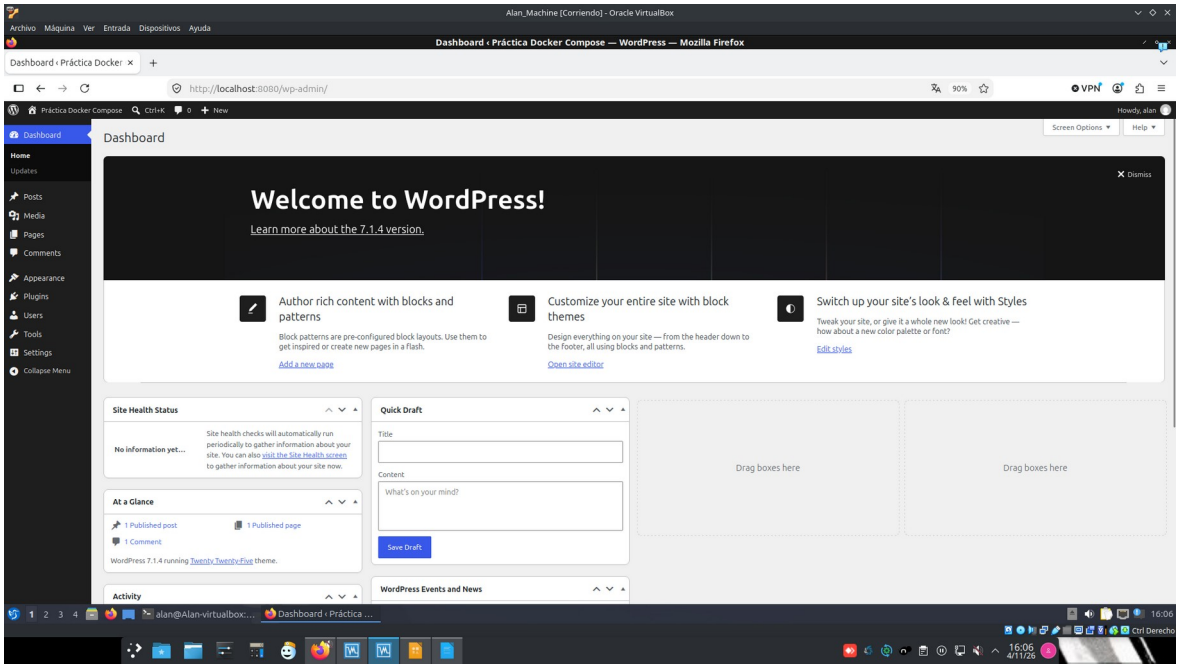


Figura 13. Panel de administración de WordPress.

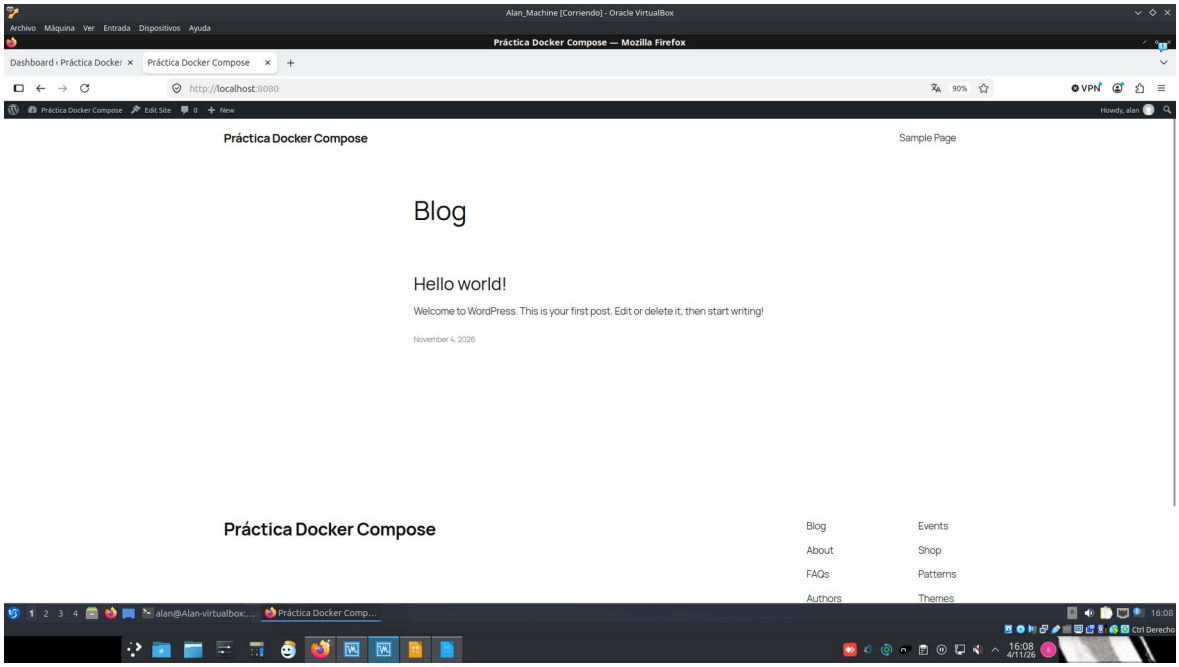


Figura 14. Página pública de WordPress en funcionamiento.

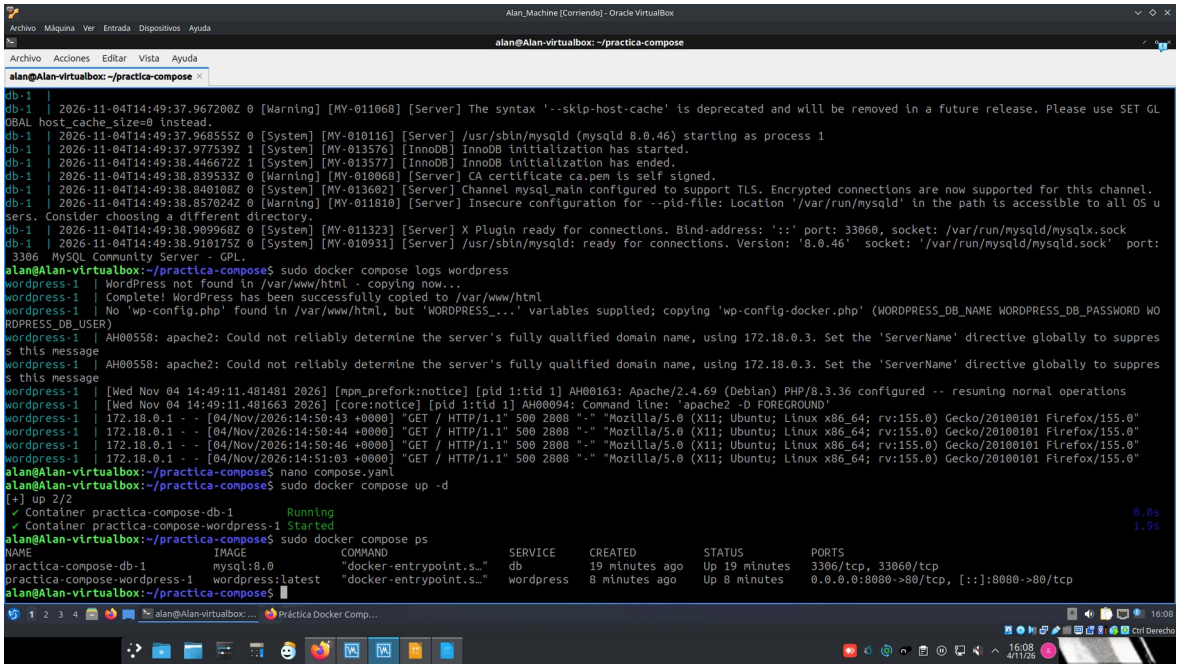


Figura 15. Comprobación final de los contenedores.

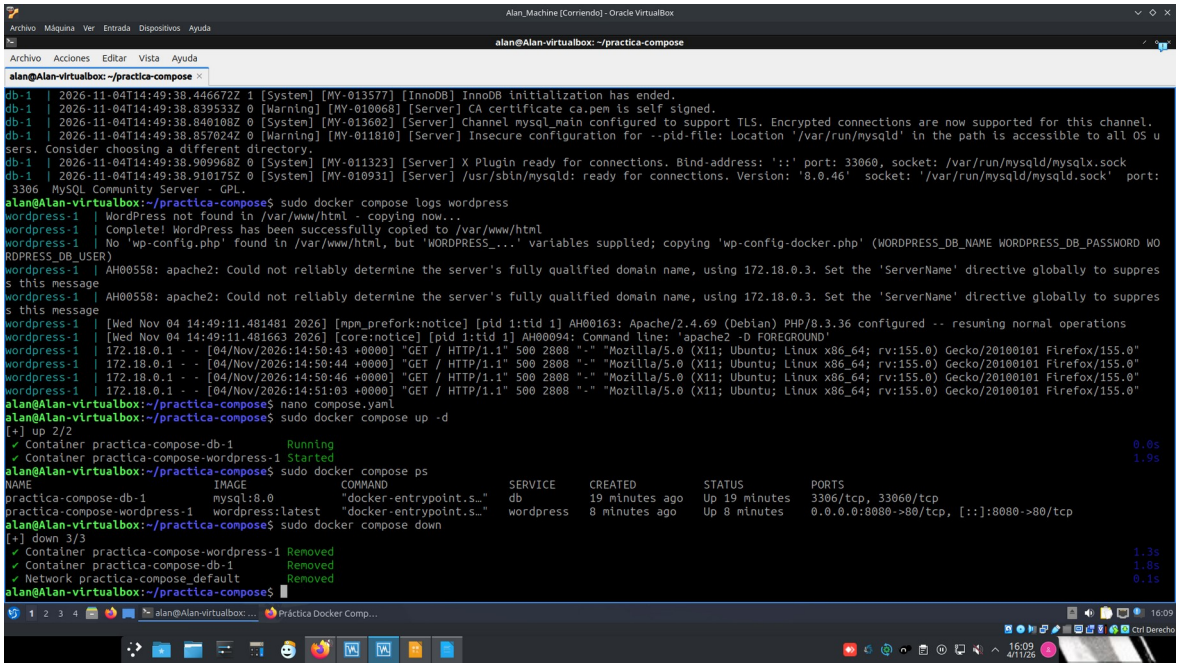


Figura 16. Detención y eliminación de los contenedores y la red.

5. Resultados y conclusiones

Las pruebas realizadas demuestran que Docker se ha instalado correctamente y que se han podido descargar, iniciar, comprobar, detener y eliminar contenedores. Mediante Docker Compose se han desplegado dos servicios relacionados, WordPress y MySQL, se ha configurado la comunicación entre ellos y se ha verificado el acceso a WordPress desde el navegador. Al finalizar, el comando docker compose down ha eliminado los contenedores y la red del proyecto.

Las actividades opcionales no se han realizado, de acuerdo con las indicaciones recibidas.

6. Actividad 03. Foro

1. ¿Cuál es la principal diferencia entre una imagen y un contenedor?

Una imagen es una plantilla que contiene todo lo necesario para ejecutar una aplicación. Un contenedor es una instancia de esa imagen que se encuentra en ejecución o preparada para ejecutarse.

2. ¿Qué comando inicia un contenedor a partir de una imagen?

El comando es `docker run`.

Ejemplo:

```
sudo docker run nginx
```

Este comando crea e inicia un contenedor utilizando la imagen de Nginx.

3. ¿Por qué un contenedor pesa mucho menos que una máquina virtual?

Porque un contenedor comparte el núcleo del sistema operativo del equipo anfitrión y no necesita ejecutar un sistema operativo completo. En cambio, una máquina virtual necesita su propio sistema operativo, lo que consume más espacio y recursos.

4. Añade cuatro comandos diferentes y explica para qué sirven, con un ejemplo de cada uno.

Comando	Función	Ejemplo
`docker pull`	Descarga una imagen de Docker Hub.	`sudo docker pull nginx`
`docker ps`	Muestra los contenedores que están en ejecución.	`sudo docker ps`
`docker stop`	Detiene un contenedor en ejecución.	`sudo docker stop servidor-practica`
`docker rm`	Elimina un contenedor detenido.	`sudo docker rm servidor-practica`

Conclusión: Docker permite ejecutar aplicaciones de forma aislada y ligera mediante contenedores, facilitando su instalación, gestión y despliegue.